SEEK: Self-certainty Evaluated Evidence-gated Knowledge-retrieval

基於自我確信評估與問題拆解的自適應知識檢索方法

Author: Hung-Yu Chien（簡宏諭）
Advisor: Prof. Hung-Yu Kao（高宏宇 教授）
July 27th, 2026

National Tsing Hua University
Intelligent Knowledge Management Lab.

Outline(未完成)

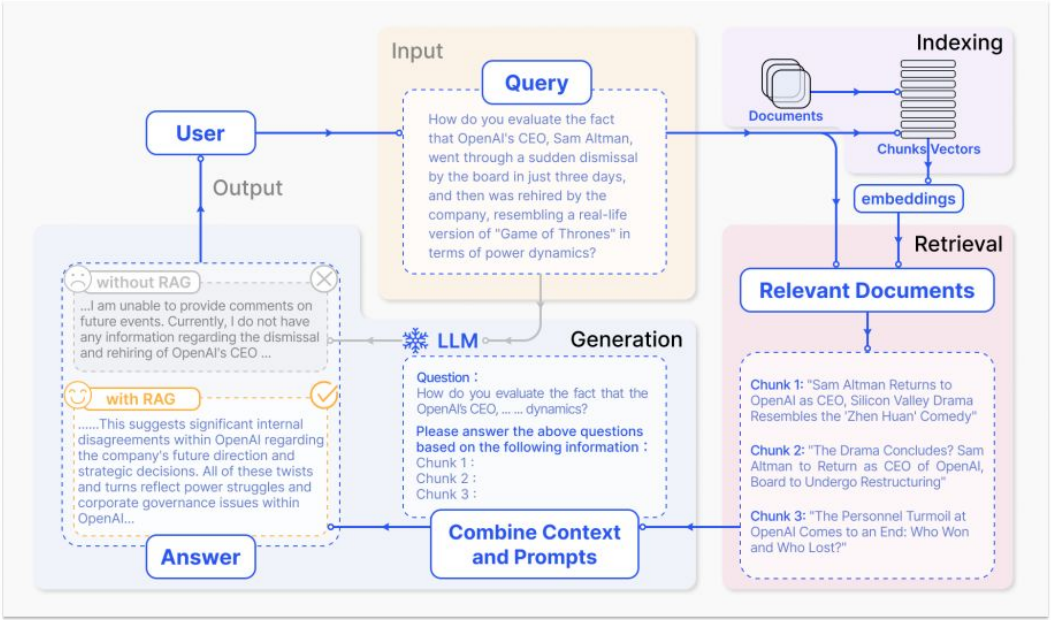
- Introduction
  - RAG motivation and limitations
  - From multi-step retrieval to adaptive RAG
  - SEEK research motivation
- Related Work
  - Adaptive-RAG
  - FLARE
  - DeepRAG
  - Comparison
- Methodology
  - SEEK framework overview
  - Data construction and decomposer training
  - Dual-path inference and rule-based verifier
- Experiments
  - Experimental setup and main results
  - Retrieval and gating ablations
  - Path-aware retrieval and evidence union
- Analysis
  - Direct-vs-decompose routing behavior
  - Oracle routing gap
- Conclusion
  - Limitations and future directions

Introduction

- **RAG motivation and limitations**
- **From multi-step retrieval to adaptive RAG**
- **SEEK research motivation**

Question Answer with RAG

Large language models often hallucinate when queried about recent, obscure, or proprietary **information outside their training scope**. Retrieval-Augmented Generation (RAG) addresses this by fetching relevant external data, **keeping the models updated and significantly reducing factually incorrect outputs**.



[1] Gao et al. "Retrieval-Augmented Generation for Large Language Models: A Survey" (arXiv 2023)

The Limitations of Standard RAG

However, applying RAG to every question creates two main problems:

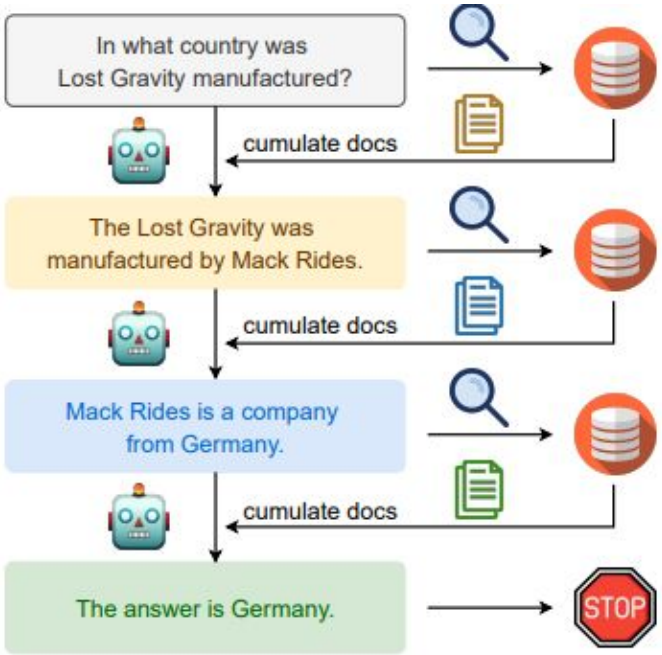
- **For simple queries:** Unnecessary retrieval just **adds noise** and confuses the model.
- **For complex queries:** A single retrieval step is often **insufficient**, thus requiring multi-step reasoning to reach an accurate conclusion.

Type	Example Dialog	Explanation
Retrieval Intent	**Retrieved context:** At high altitudes, water boils at lower temperatures than at sea level, such as 95°C at elevations above 2,000 meters. This altitude-induced reduction can affect cooking times and methods. **Query:** What is the boiling point of water at sea level? **Response:** The boiling point of water at sea level is 95°C.	**This query involves a basic scientific fact that should be available within the model's parametric knowledge. However, the system inappropriately retrieves** and uses altitude-specific information about boiling temperatures, leading to an incorrect response for sea-level conditions.
Complex Queries	**Retrieved context:** Alexander Fleming, known for his groundbreaking work in discovering penicillin, significantly influenced modern antibiotics. Fleming's discovery saved countless lives. **Query:** Which university did the scientist who discovered penicillin graduate from? **Response:** Alexander Fleming graduated from Oxford University.	**The retrieval system only manages to fetch information about Fleming's professional achievements in the discovery of penicillin**. However, the document does not provide information about his educational background, thus the model generates a hallucinatory answer.

[2] Gao et al. "A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open estions"(ACM Transactions on Information Systems 2025)

The Rise of Multi-Step Retrieval

To address complex queries, researchers introduced **multi-step retrieval**, allowing models to iteratively fetch and reason over information.

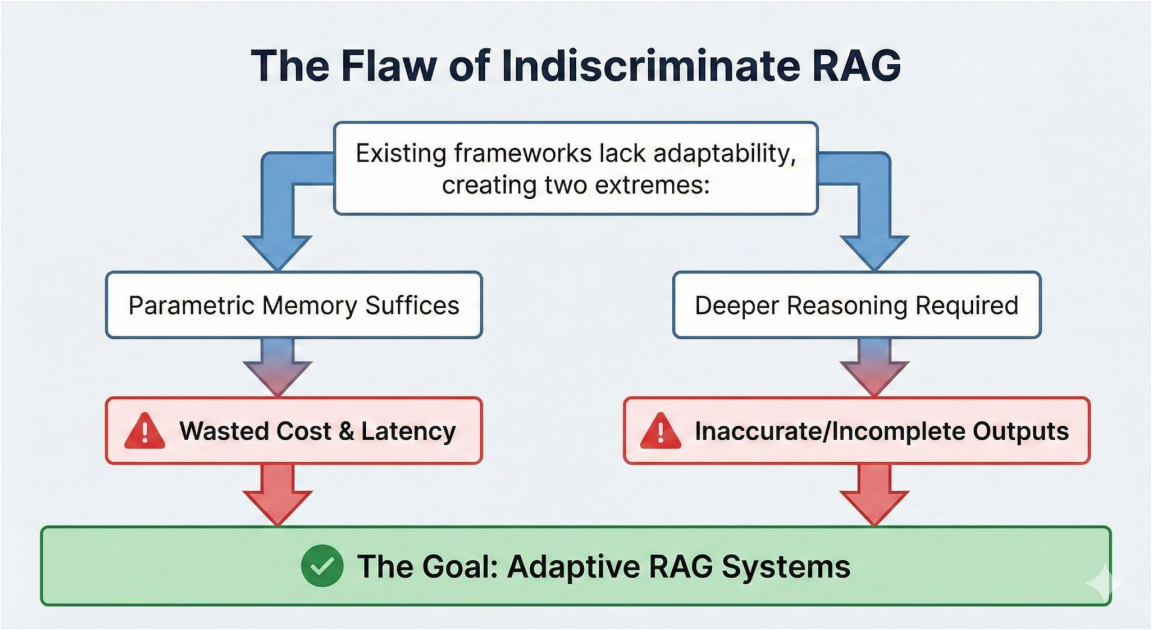


[3] Trivedi et al. "Interleaving Retrieval with Chain-of-Thought Reasoning for Knowledge-Intensive Multi-Step Questions"(ACL 2023)

The Need for Adaptive RAG

Existing RAG applies retrieval indiscriminately, lacking flexibility.

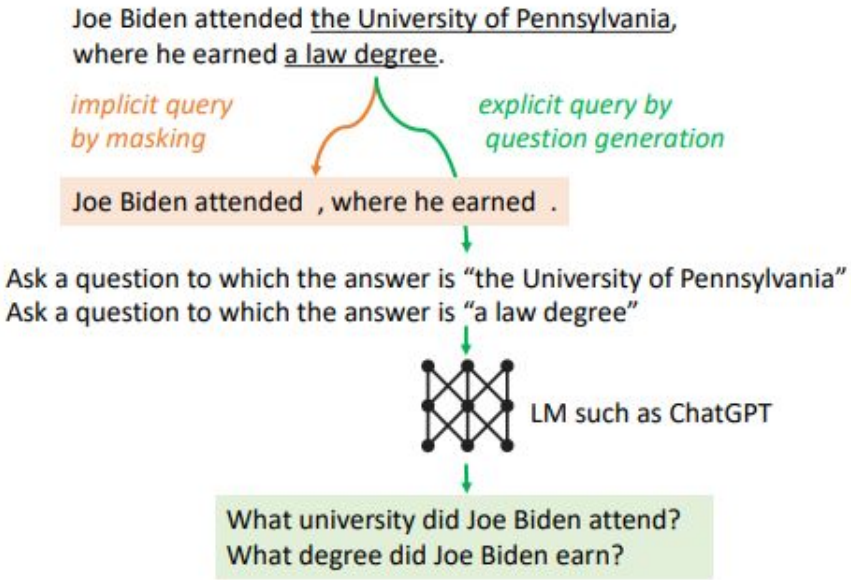
- Unnecessary retrieval spikes **latency and cost**.
- Insufficient retrieval yields **incorrect answers**.



[4] Su et al. "Fast or Better? Balancing Accuracy and Cost in RetrievalAugmented Generation with Flexible User Control"(CoRR 2025)

Recent Works in Adaptive RAG

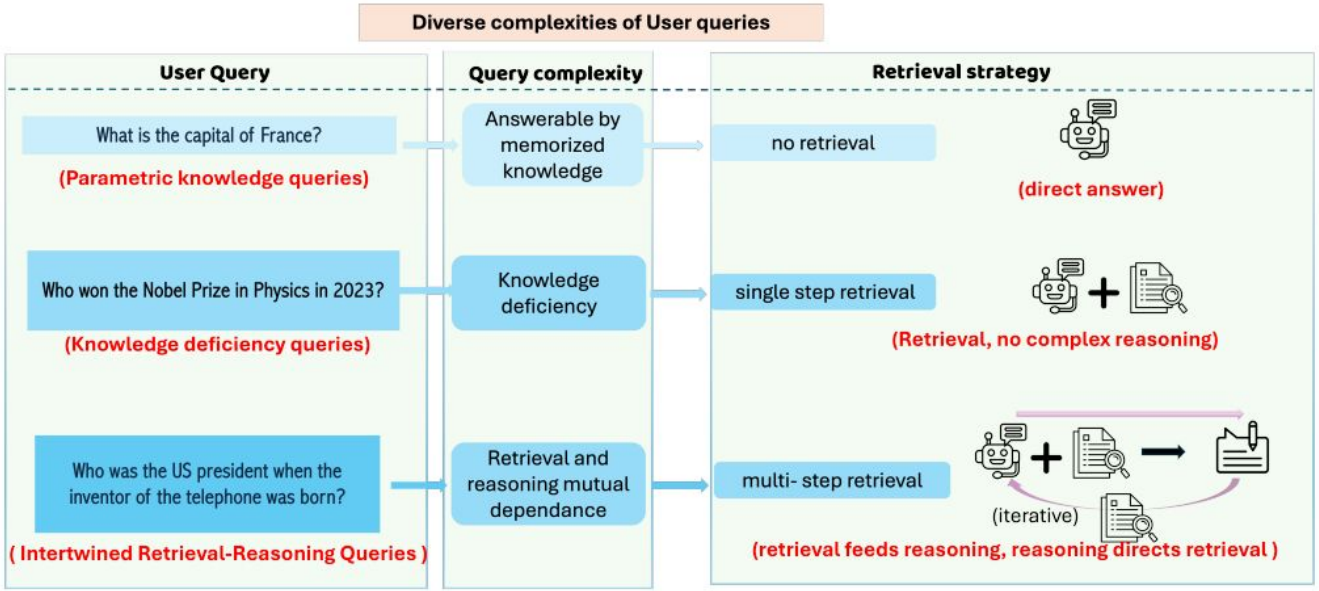
- Active Retrieval : Methods like SeaKR[5] and FLARE[6] trigger retrieval only when necessary to avoid noise.
- Complexity-Aware Routing : Adaptive-rag[7] recognizes that query difficulty varies significantly in the real world.



[5] Yao et al. "SeaKR: Self-aware Knowledge Retrieval for Adaptive Retrieval Augmented Generation"(ACL 2025)
[6] Jiang et al. "Active Retrieval Augmented Generation (FLARE)"(EMNLP 2023)
[7] Jeong et al. "Adaptive-rag: Learning to adapt retrieval-augmented large language models through question complexity"(NAACL 2024)

Recent Works in Adaptive RAG

- Active Retrieval : Methods like SeaKR[5] and FLARE[6] trigger retrieval only when necessary to avoid noise.
- Complexity-Aware Routing : Adaptive-rag[7] recognizes that query difficulty varies significantly in the real world.



[5] Yao et al. "SeaKR: Self-aware Knowledge Retrieval for Adaptive Retrieval Augmented Generation"(ACL 2025)
[6] Jiang et al. "Active Retrieval Augmented Generation (FLARE)"(EMNLP 2023)
[7] Jeong et al. "Adaptive-rag: Learning to adapt retrieval-augmented large language models through question complexity"(NAACL 2024)

Motivation



1. Multi-step retrieval improves complex question answering by decomposing a question into smaller sub-questions. However, **decomposition is not always reliable**: it may remove useful context.



2. This motivates a decomposition-routing view of adaptive RAG, where the system decides not only **when to retrieve**, but also **when to trust decomposition**.



3. We aim to build a lightweight sub-question-level adaptive RAG framework that **controls retrieval while preserving the original-question path**.

SEEK

We introduce **SEEK(Self-certainty Evaluated Evidence-gated Knowledge-retrieval)** as a lightweight adaptive retrieval framework for question answering.

1. **Dual-path answering**
   Preserve both direct and decomposed answers.

2. **Evidence-gated sub-question retrieval**
   Retrieve only when a sub-question answer is uncertain.

3. **Original-question context preservation**
   Add original-question evidence back to decomposed retrieval.

4. **Training-free verification**
   Select the final answer using different signals.

Related Work

- **Adaptive-RAG**
- **FLARE**
- **DeepRAG**
- **Comparison**

FLARE

- Standard RAG just does **one search when you ask a question.** That works for short replies, but it struggles with long, detailed answers because the model needs more information to answer.

Two Guiding Principles

WHEN to retrieve

Only when the LM is uncertain about what to say next — avoid unnecessary or inappropriate retrieval.

WHAT to retrieve

Queries should reflect future content — look ahead at what the LM is about to say, not just what it has said.

Core Idea

Iteratively **predict the next sentence** as a look-ahead probe. If it contains **low-probability tokens**, treat them as a knowledge gap → retrieve and regenerate. Otherwise, commit and continue.

[6] Jiang et al. "Active Retrieval Augmented Generation (FLARE)"(EMNLP 2023)

FLARE-direct: Iterative Loop

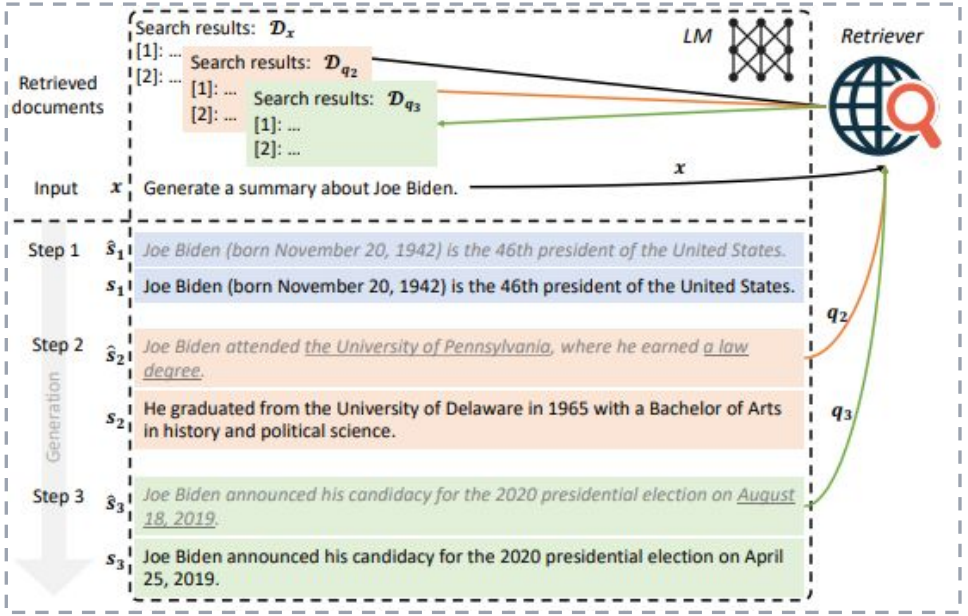
- At each step, FLARE-direct runs a **generate → detect → retrieve → regenerate** loop, triggered purely by token probabilities.

1 Predict next sentence
Generate a tentative sentence ŷ from current context.

2 Detect low-confidence tokens
If min token prob < θ, mark span as uncertain.

3 Retrieve with look-ahead
Form query from ŷ → fetch docs D from retriever.

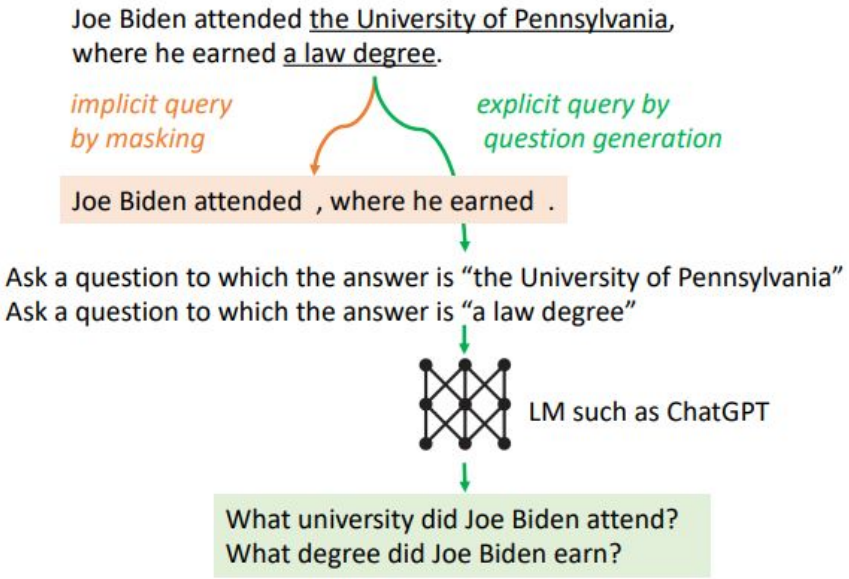
4 Regenerate sentence
Condition on D, regenerate sentence, commit, advance.



[6] Jiang et al. "Active Retrieval Augmented Generation (FLARE)"(EMNLP 2023)

FLARE : Query Formulation

- Once low-probability tokens are detected, how do we **turn them into a retrieval query**? FLARE proposes two strategies.



Implicit · Masking

Mask out low-probability tokens; use the remaining sentence as the query.

Example: *"Joe Biden attended ___, where he earned ___."*

Explicit · Question Generation

Prompt an LM to write a question whose answer is the low-probability span.

Example: *"What university did Joe Biden attend?"*

[6] Jiang et al. "Active Retrieval Augmented Generation (FLARE)"(EMNLP 2023)

Adaptive-RAG: Overview

- Real-world queries vary in difficulty — but most RAG systems apply a one-size-fits-all retrieval strategy. Simple queries get wasted compute; complex ones get insufficient evidence.

Why One Strategy Doesn't Fit All

① Always retrieve → high cost

Simple factual queries the LM already knows are sent through the retriever — extra latency and noise for no gain.

② Single retrieval → insufficient

Multi-hop questions need iterative retrieval to chain evidence; a single pass under-retrieves and yields wrong answers.

Core Idea

Route each query to the right retrieval strategy. Train a **small classifier** that predicts query complexity, then dispatch to **no / single-step / multi-step retrieval** — spending compute only where it's needed.

[7] Jeong et al. "Adaptive-rag: Learning to adapt retrieval-augmented large language models through question complexity"(NAACL 2024)

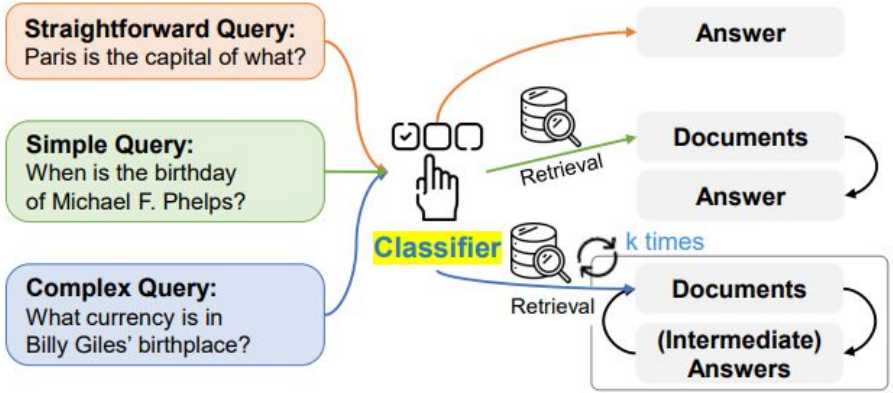
Adaptive-RAG: Three Retrieval Strategies

- Adaptive-RAG unifies **three retrieval paradigms** under one framework, picking one per query based on predicted complexity.

A — No retrieval
LM answers directly from parametric memory — cheapest path, for queries it already knows.

B — Single-step retrieval
One retrieve + one generate — covers most factoid queries needing external evidence.

C — Multi-step retrieval
Iterative retrieve + reason loop — handles multi-hop chains across documents.



[7] Jeong et al. "Adaptive-rag: Learning to adapt retrieval-augmented large language models through question complexity"(NAACL 2024)

Adaptive-RAG: The Complexity Classifier

- The router is a **small T5-Large classifier** predicting one of three labels — trained **without human annotation**.

Classifier I/O

Input	The query string.
Model	T5-Large — 770M.
Output	One of three labels: A (no retrieval), B (single-step), C (multi-step).
Inference	Run once per query; result determines which downstream pipeline executes.

Silver labels · from model outcomes

Run each query through all three strategies; assign the simplest one that produces the correct answer as the label.

Example: if both A and B answer correctly, label = A (prefer cheaper).

Dataset bias · for unlabeled queries

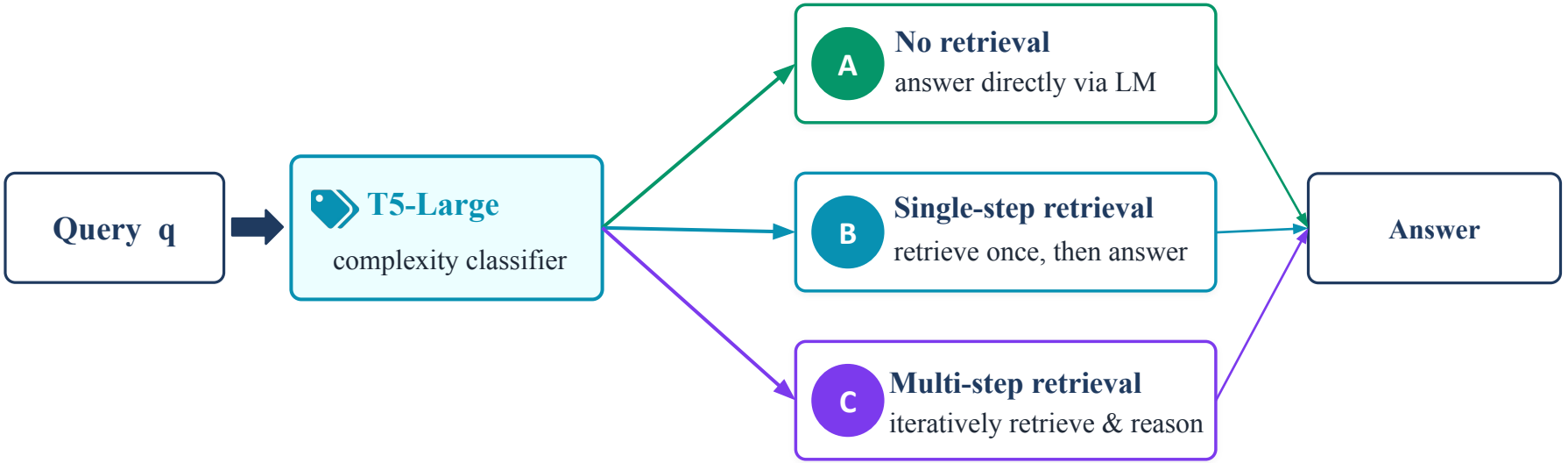
If silver labelling fails, fall back on the source dataset's design: single-hop QA → B, multi-hop QA → C.

Example: HotpotQA / MuSiQue queries default to label C; NQ defaults to B.

[7] Jeong et al. "Adaptive-rag: Learning to adapt retrieval-augmented large language models through question complexity"(NAACL 2024)

Adaptive-RAG: End-to-End Pipeline

- At inference time, each query passes through the classifier **once**, then takes exactly one of three downstream paths.



[7] Jeong et al. "Adaptive-rag: Learning to adapt retrieval-augmented large language models through question complexity"(NAACL 2024)

DeepRAG

- Standard RAG retrieves **every step** or **not at all** — inflexible for multi-hop reasoning, where **some sub-questions need retrieval, others don't**.

Two Problems with Existing Iterative RAG

① Poor task decomposition

LMs struggle to generate atomic, precise sub-queries — decomposition is often ambiguous or redundant.

② Redundant retrieval

Retrieving at every step injects noise and latency; some sub-questions are answerable from parametric memory alone.

Core Idea

Model retrieval-augmented reasoning as a **Markov Decision Process**. Decompose the question into sub-queries; for each one, make an **atomic decision** — parametric memory or retriever — driven by awareness of the model's **knowledge boundaries**.

[8] Guan et al. "DEEPRAG: THINKING TO RETRIEVE STEP BY STEP FOR LARGE LANGUAGE MODELS"(ICLR 2026)

DeepRAG: Three-Phase Pipeline

- DeepRAG builds the adaptive policy through **three sequential phase** that synthesize data → imitate → calibrate.

1 Binary Tree Search

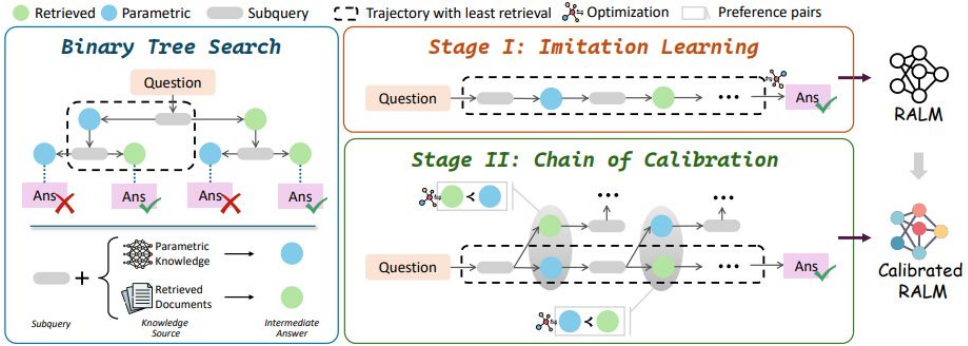
For each sub-query, expand a binary tree over {retrieve, parametric}; find the path that yields the correct answer with minimum retrievals.

2 Imitation Learning

SFT on the best paths from Stage 1 — teach the LM the pattern of "sub-query → atomic decision → intermediate answer".

3 Chain of Calibration

Use binary tree search to build preference pairs (good vs bad atomic decisions); refine with DPO to calibrate knowledge-boundary awareness.



[8] Guan et al. "DEEPRAG: THINKING TO RETRIEVE STEP BY STEP FOR LARGE LANGUAGE MODELS"(ICLR 2026)

Summary of Related Works

Method	Decision Unit	Decision Signal	Takeaway
Adaptive-RAG	Question	Classifier	Too coarse for multi-step reasoning
FLARE	Token / Sentence	Token probability	Too noisy for sub-question reasoning
DeepRAG	Sub-question	Learned policy	Strong but heavy and less transparent
SEEK (Ours)	**Sub-question**	**Entropy + Verifier**	**Lightweight and interpretable**

Prior work shows that adaptive retrieval is necessary, but the retrieval decision should happen at the **sub-question level** with a **lightweight and interpretable signal**.

Methodology

- **SEEK framework overview**
- **Data construction and decomposer training**
- **Dual-path inference and rule-based verifier**

Separate Entropy into Head and Tail

Partition the distribution into head and tail to separate mass allocation from internal uncertainty.

$$H(p) = -\sum_{v \in \mathcal{V}} p(v) \log p(v).$$

$$m_{\text{head}} = \sum_{v \in \mathcal{V}_{\text{head}}} p(v).$$

$$p_{\text{head}}(v) = p(v \mid v \in \mathcal{V}_{\text{head}}) = \frac{p(v)}{m_{\text{head}}}, \qquad v \in \mathcal{V}_{\text{head}},$$

$$p_{\text{tail}}(v) = p(v \mid v \in \mathcal{V}_{\text{tail}}) = \frac{p(v)}{1 - m_{\text{head}}}, \qquad v \in \mathcal{V}_{\text{tail}}.$$

Separate Entropy into Head and Tail

Partition the distribution into head and tail to separate mass allocation from internal uncertainty.

$$H(p) = -\sum_{v \in \mathcal{V}} p(v) \log p(v).$$

$$H(p) = -\sum_{v \in \mathcal{V}_{\text{head}}} p(v) \log p(v) - \sum_{v \in \mathcal{V}_{\text{tail}}} p(v) \log p(v)$$

$$= -\sum_{v \in \mathcal{V}_{\text{head}}} m_{\text{head}} p_{\text{head}}(v) \log \left[m_{\text{head}} p_{\text{head}}(v) \right]$$

$$- \sum_{v \in \mathcal{V}_{\text{tail}}} (1 - m_{\text{head}}) \, p_{\text{tail}}(v) \log \left[(1 - m_{\text{head}}) \, p_{\text{tail}}(v) \right]$$

$$= -m_{\text{head}} \log m_{\text{head}} - (1 - m_{\text{head}}) \log (1 - m_{\text{head}})$$

$$+ m_{\text{head}} H(p_{\text{head}}) + (1 - m_{\text{head}}) \, H(p_{\text{tail}})$$

Head/Tail Mass Is Most Predictive

Error-detection AUROC, average across four datasets

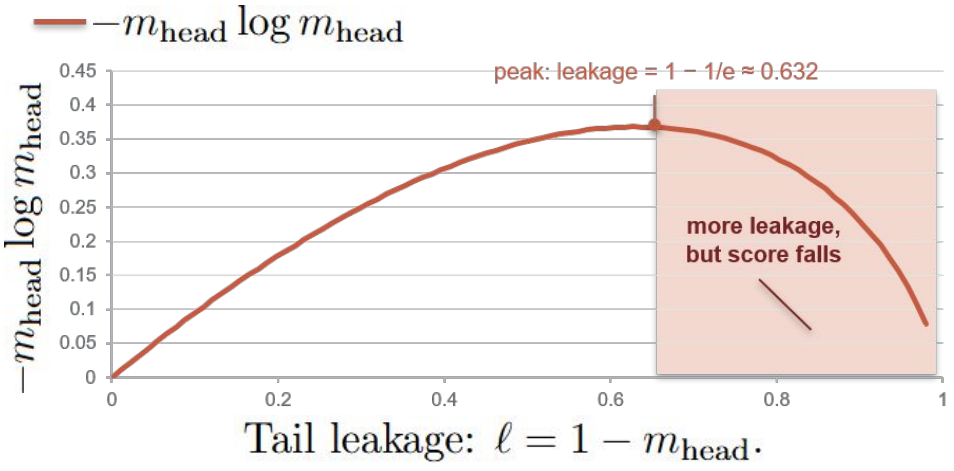
$$
\begin{aligned}
H(p) &= -\sum_{v \in \mathcal{V}} p(v) \log p(v). \\
&= -m_{\text{head}} \log m_{\text{head}} - (1 - m_{\text{head}}) \log (1 - m_{\text{head}}) \\
&\quad + m_{\text{head}} H(p_{\text{head}}) + (1 - m_{\text{head}}) H(p_{\text{tail}})
\end{aligned}
$$

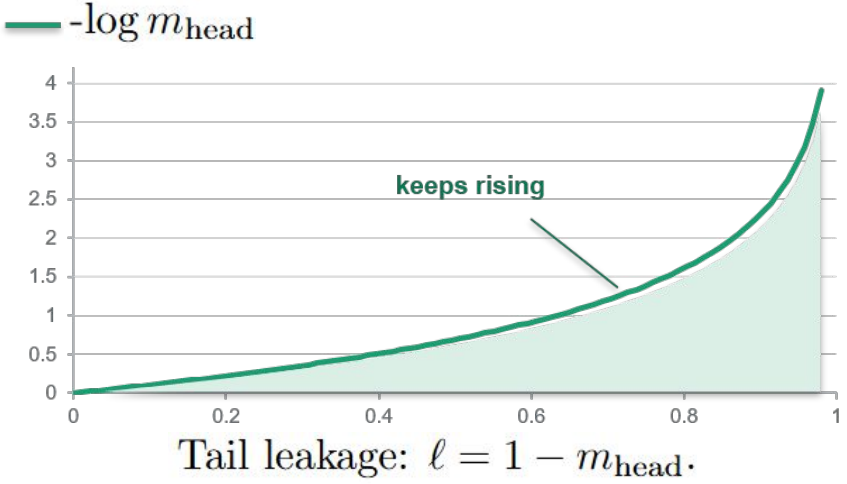
$-m_{\text{head}} \log m_{\text{head}} - (1 - m_{\text{head}}) \log (1 - m_{\text{head}})$	**: Head Retained-Mass Signal**	**0.742**
$H(p_{\text{head}})$	**: Uncertainty within head**	0.712
$H(p_{\text{tail}})$	**: Uncertainty within tail**	0.502

Why Use -log m_{head}

$-\log m_{\text{head}}$ increases as more probability leaks to the tail.

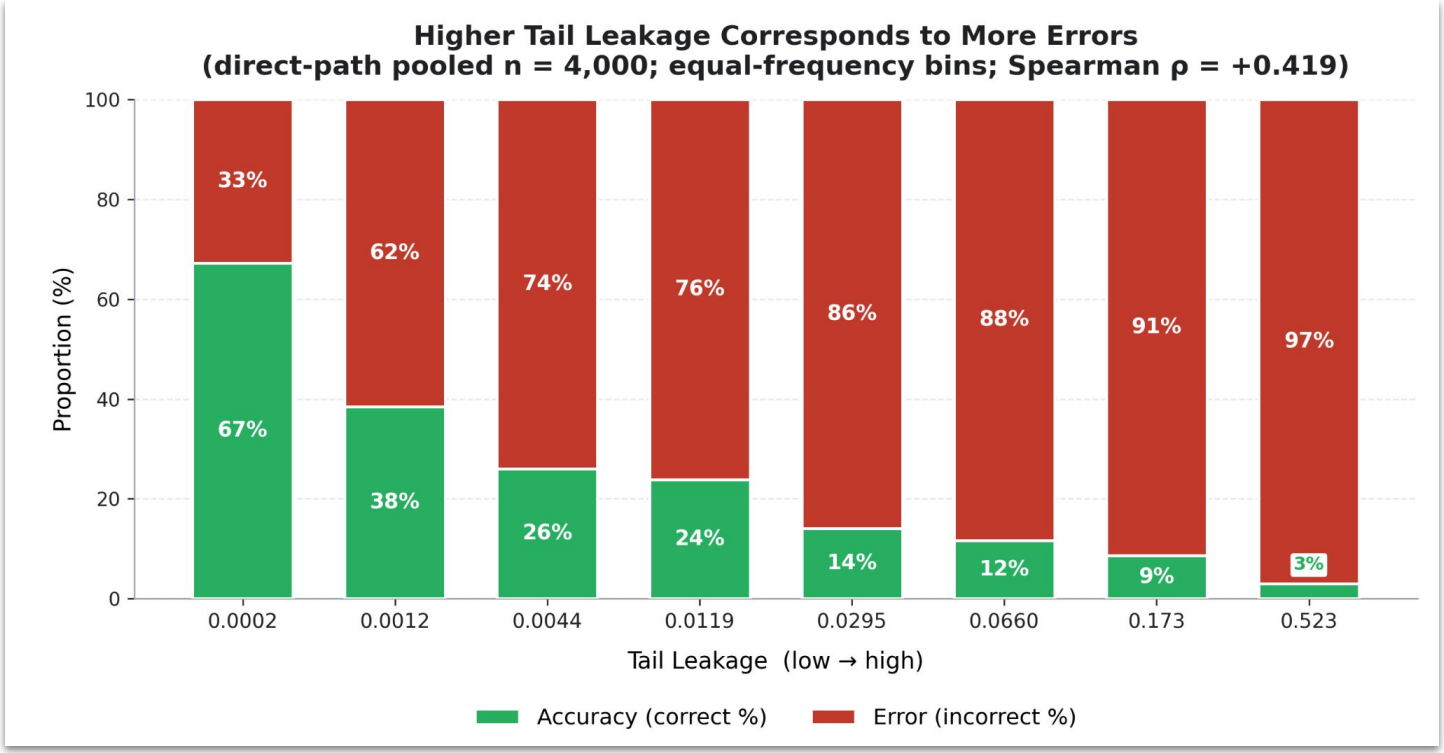
$$-m_{\text{head}} \log m_{\text{head}} - (1 - m_{\text{head}}) \log (1 - m_{\text{head}})$$





Higher Tail Leakage Predicts More Errors

Higher Tail-Leakage Scores correspond to higher error rates.

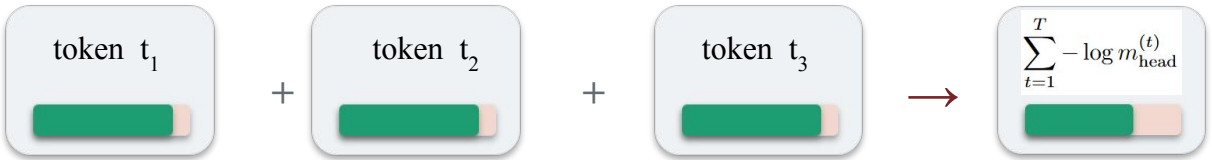


Aggregating Leakage Across the Answer

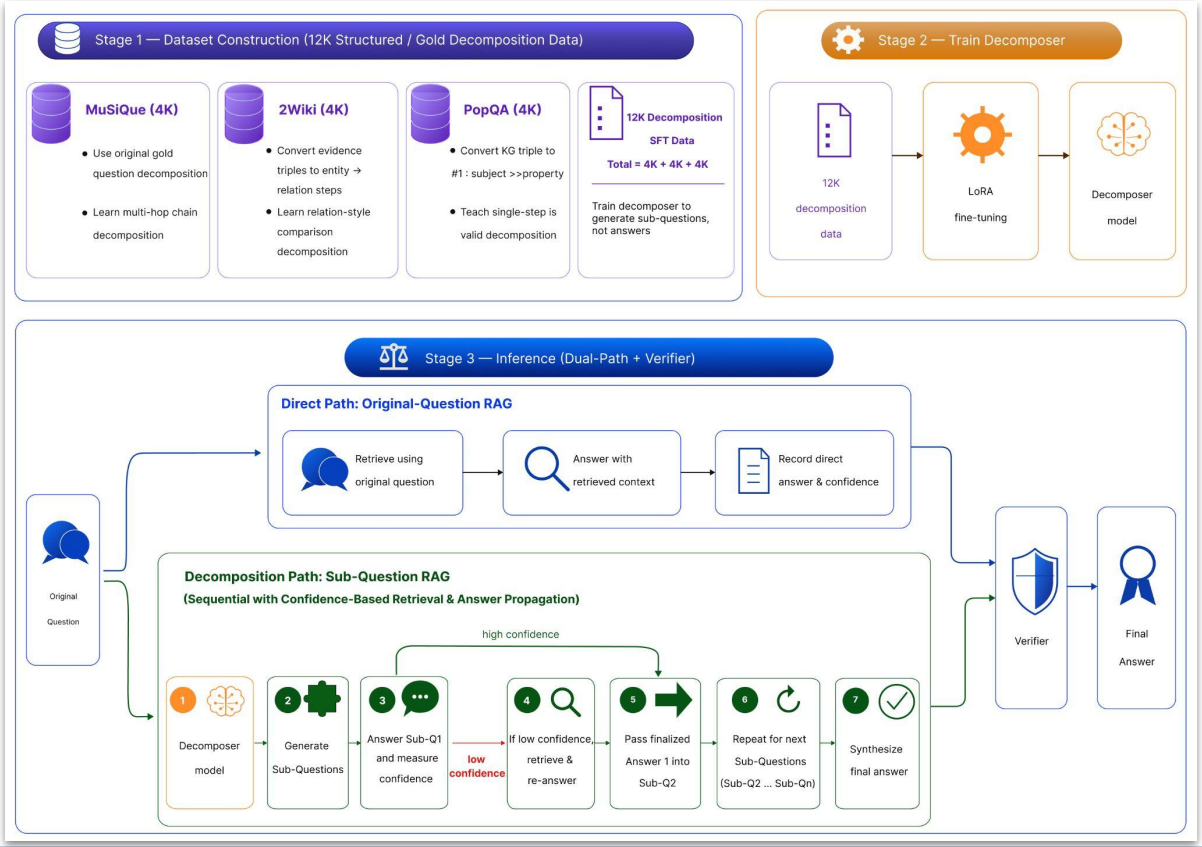
Sum $-\log m_{\text{head}}^{(t)}$ across tokens to capture small leaks distributed throughout the answer.

$$
\begin{aligned}
H(p) &= -\sum_{v \in \mathcal{V}} p(v) \log p(v). \\
&= -m_{\text{head}} \log m_{\text{head}} - (1 - m_{\text{head}}) \log (1 - m_{\text{head}}) \\
&\quad + m_{\text{head}} H(p_{\text{head}}) + (1 - m_{\text{head}}) H(p_{\text{tail}})
\end{aligned}
$$

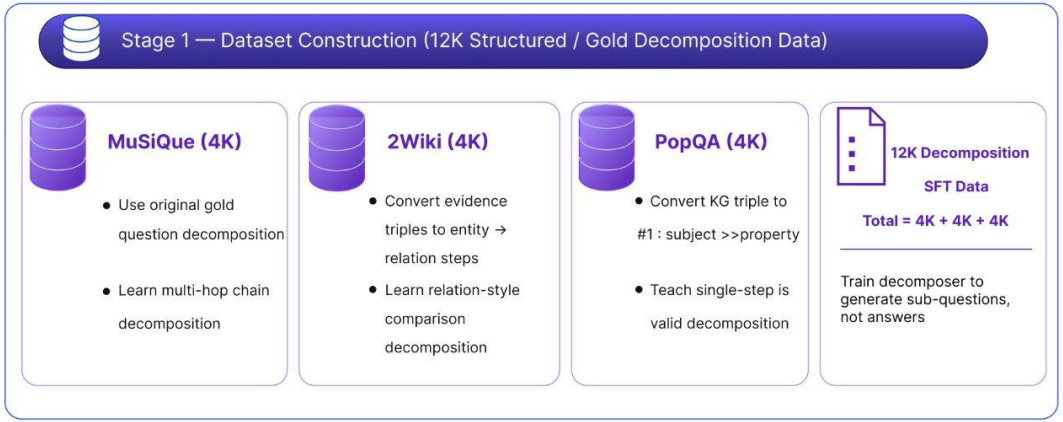
$$
\longrightarrow \; -\log m_{\text{head}}^{(t)} \qquad \text{TailLeakage}(y) = \sum_{t=1}^{T} -\log m_{\text{head}}^{(t)}
$$



SEEK Overview



Method - Data Construction



Original Question : When was the institute that owned The Collegian founded?

① Original question_decomposition

step id	question_decomposition	answer
482757	The Collegian >> owned by	Houston Baptist Univ.
12019	When was #1 founded?	1960

② Final SFT Format

#1: The Collegian >> owned by
#2: When was #1 founded?

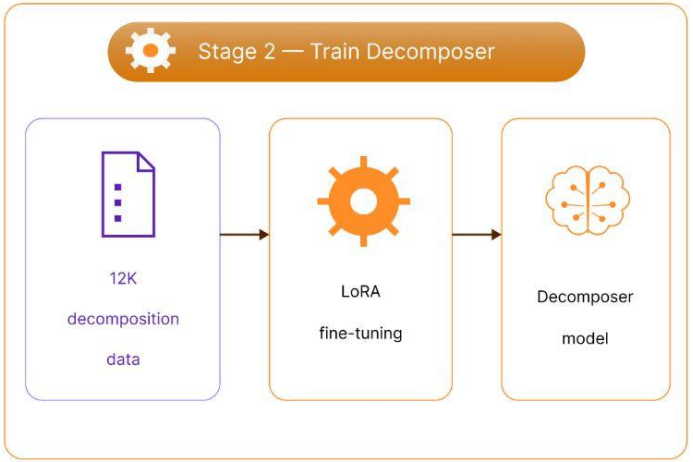
Method - Training Stage

Learn to decompose each input question sub-questions.

Input
• Use the constructed 12K decomposition dataset

Learning Target
• Learn how to split questions into sub-questions
• Use #N markers to connect dependent steps

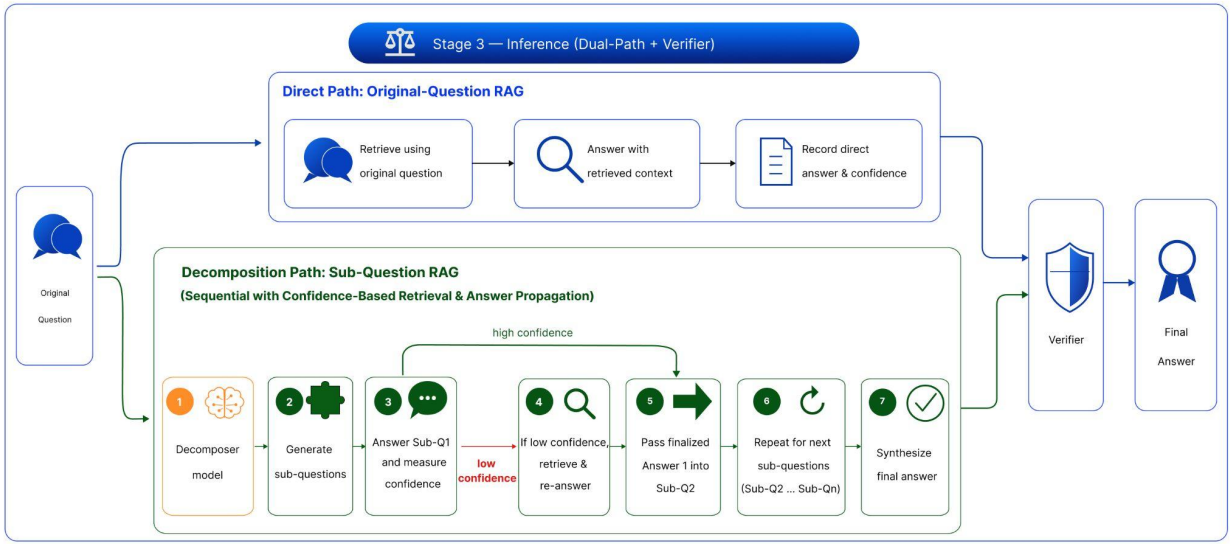
Output
• A trained decomposer model
• Generates sub-question plans for inference



Method - Inference Stage

Three Components

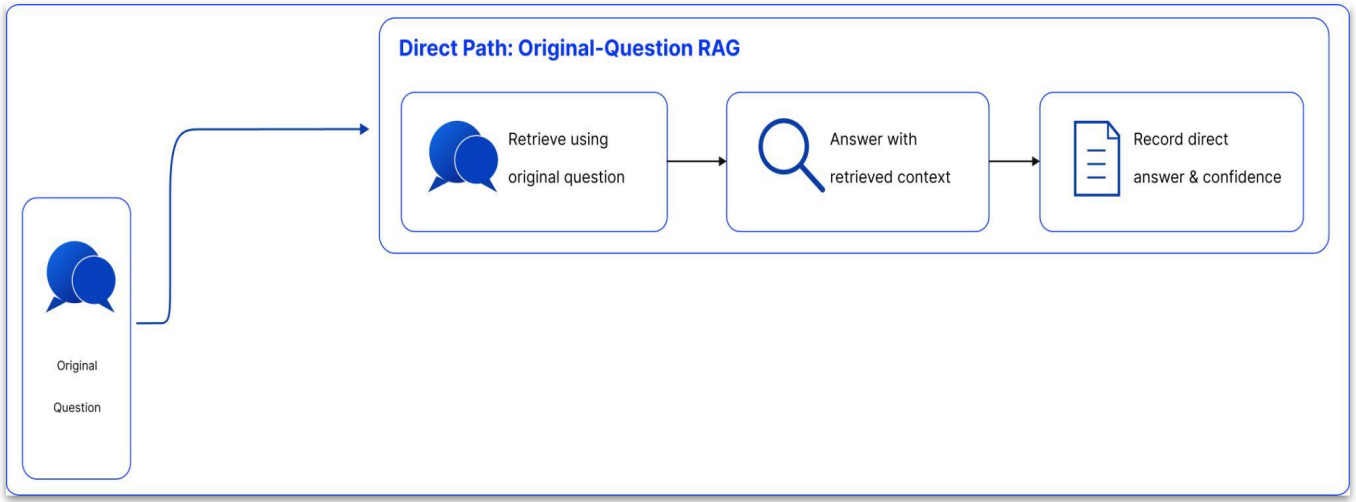
1. **Direct Path**
   Answer from the original question.
2. **Decomposition Path**
   Break the question into sub-questions and answer step by step.
3. **Rule-based Verifier**
   Select the final answer using confidence and consistency signals.



Method - Inference Stage

Preserve the original question without decomposition.

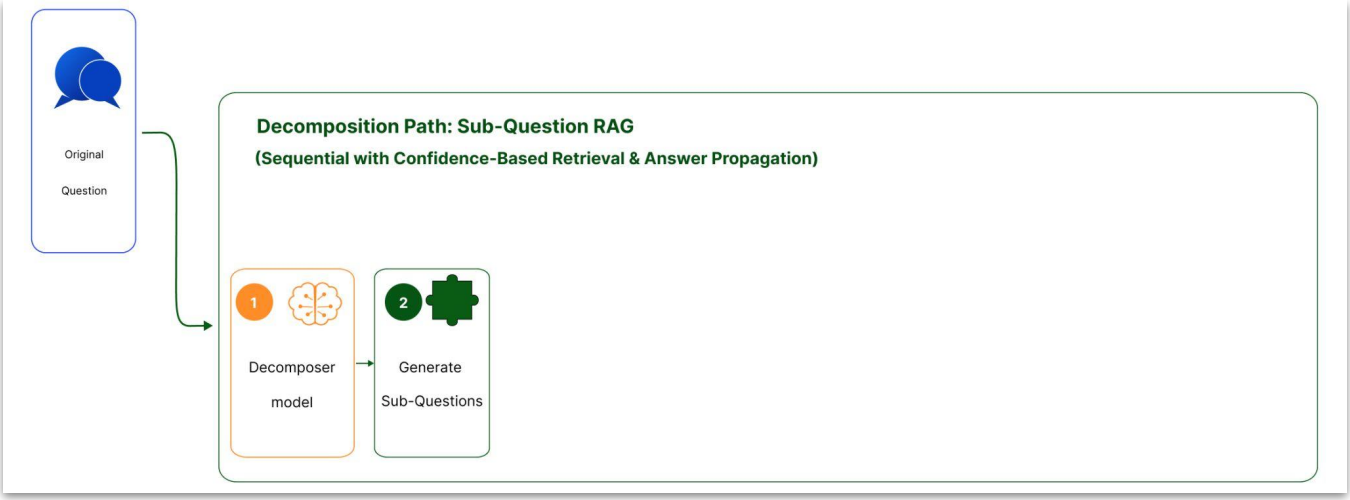
- Use the original question directly
- Generate one direct candidate answer
- Record uncertainty for verifier routing



Method - Inference Stage

Break a complex question into smaller sub-questions and solve them step by step.

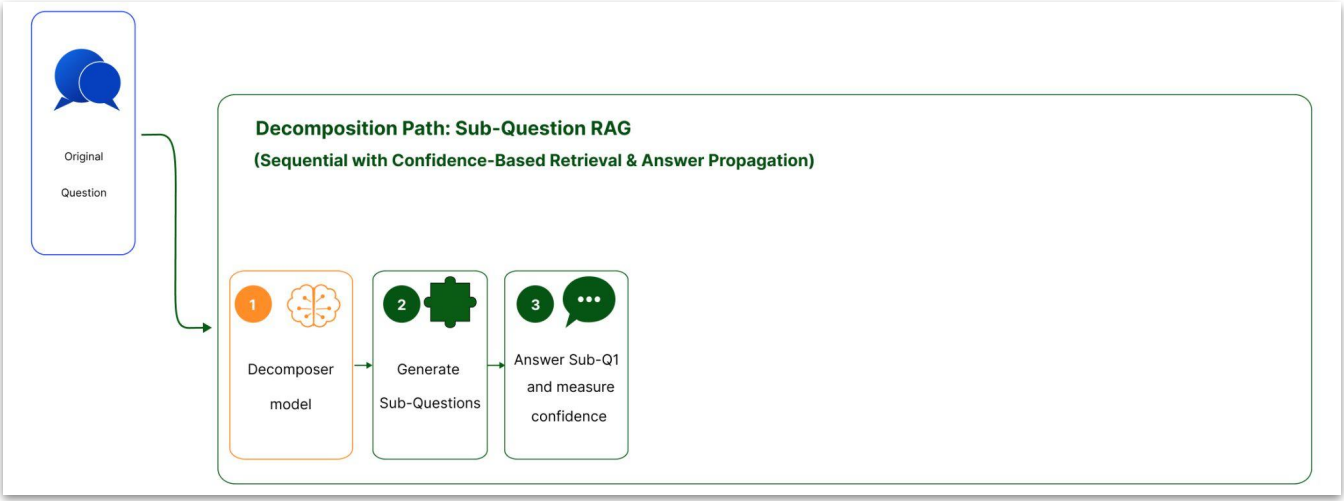
- Splits the original question into executable Sub-Questions
- Answers Sub-Q1 and measure evidence
- Retrieves evidence when a Sub-Answer is uncertain



Method - Inference Stage

Break a complex question into smaller sub-questions and solve them step by step.

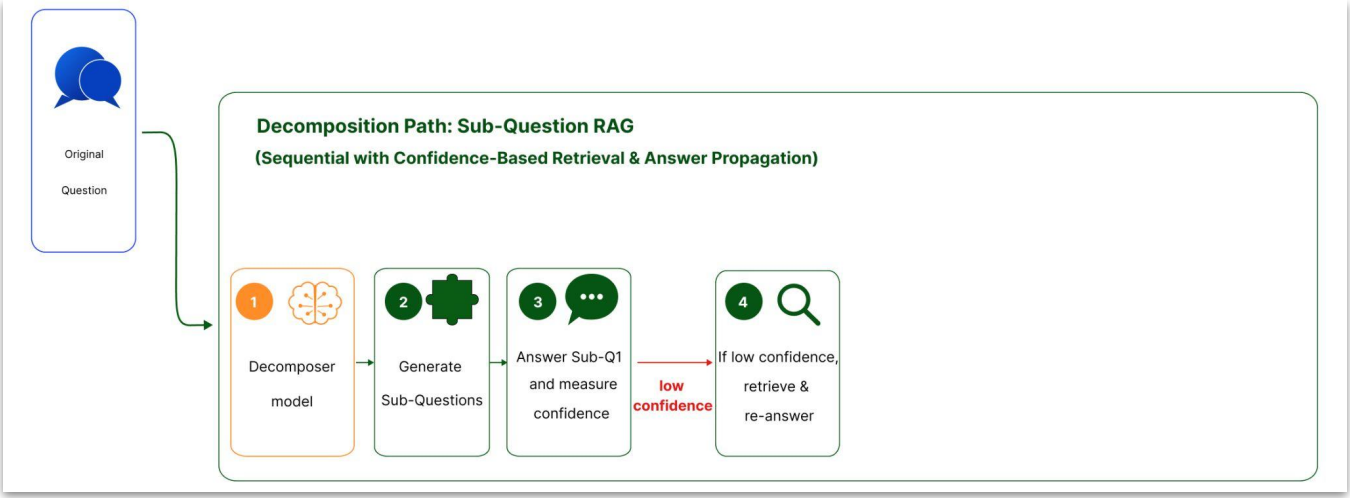
- Splits the original question into executable Sub-Questions
- Answers Sub-Q1 and measure evidence
- Retrieves evidence when a Sub-Answer is uncertain



Method - Inference Stage

Break a complex question into smaller sub-questions and solve them step by step.

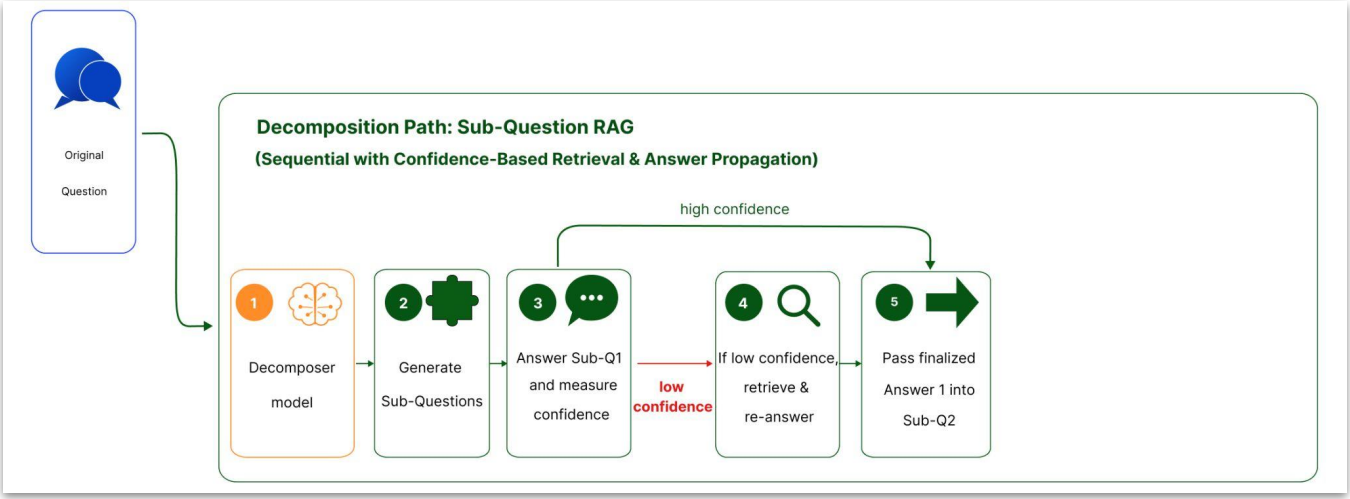
- Splits the original question into executable Sub-Questions
- Answers Sub-Q1 and measure evidence
- Retrieves evidence when a Sub-Answer is uncertain



Method - Inference Stage

Break a complex question into smaller sub-questions and solve them step by step.

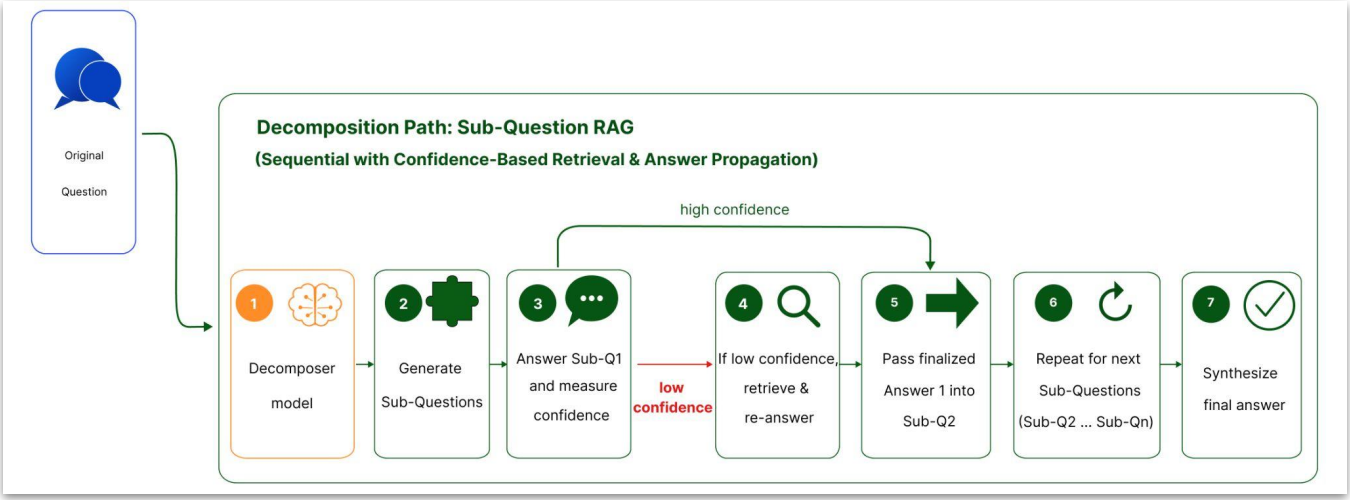
- Pass re-answer or high confidence answer into Sub-Q2
- Process Remaining Sub-questions and Synthesize Final Answer



Method - Inference Stage

Break a complex question into smaller sub-questions and solve them step by step.

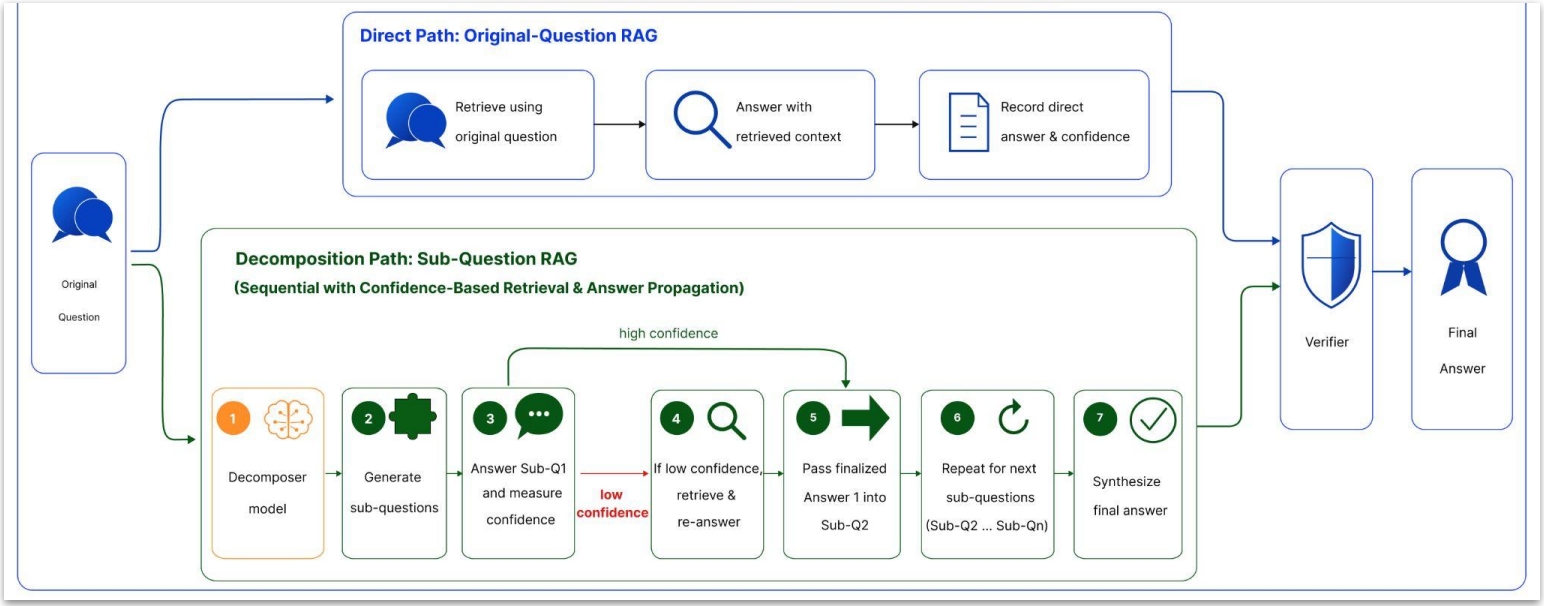
- Pass re-answer or high confidence answer into Sub-Q2
- Process Remaining Sub-questions and Synthesize Final Answer



Method - Inference Stage

The direct path is selected only under the following two conditions:

1. The answer to the original question is confident.
2. The decomposed sub-questions remain uncertain even after retrieval.



Experiment

- **Experimental setup and main results**
- **Retrieval and gating ablations**
- **Path-aware retrieval and evidence union**

Experiment Setup

- **Backbone Model** : Llama-3-8B-Instruct
- **Decomposer** : LoRA SFT decomposer
- **SFT Data** : 12K decomposition examples: MuSiQue / 2Wiki / PopQA, 4K each
- **LoRA Setting** : rank=16, alpha=32, dropout=0.05, 3 epochs
- **Evaluation Datasets** : HotpotQA、2WikiMultihopQA、MuSiQue、PopQA
- **Metrics** : Exact Match (EM)、Token-level F1
- **Compared Methods:**
  - CoT
  - FLARE
  - DeepRAG

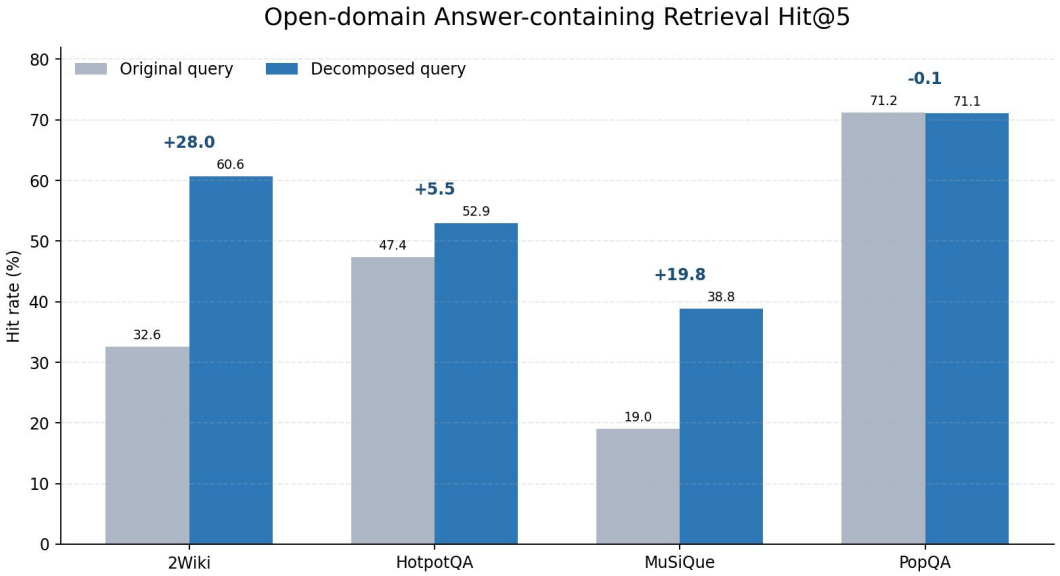
Main Result

- SEEK achieves the best average result across the four datasets, improving over DeepRAG by +3.43 EM and +3.15 F1.
- The improvement mainly comes from 2Wiki, MuSiQue, and PopQA, with the largest gain on PopQA.

Llama-3-8B-Instruct										
Method	PopQA		Hotpot QA		2WikiMultihopQA		MuSiQue		Avg	
	EM	F1	EM	F1	EM	F1	EM	F1	EM	F1
CoT	21.20	25.33	27.20	37.75	28.20	34.85	13.70	22.97	22.58	30.23
FLARE	28.80	40.61	23.80	32.88	30.30	37.45	14.50	23.57	24.35	33.63
DeepRAG	42.50	47.80	**40.70**	**51.54**	48.10	53.25	22.50	30.40	38.45	45.75
SEEK (Ours)	**53.90**	**55.73**	37.80	49.03	**49.30**	**56.22**	**26.50**	**34.63**	**41.88**	**48.90**

Retrieval Recall

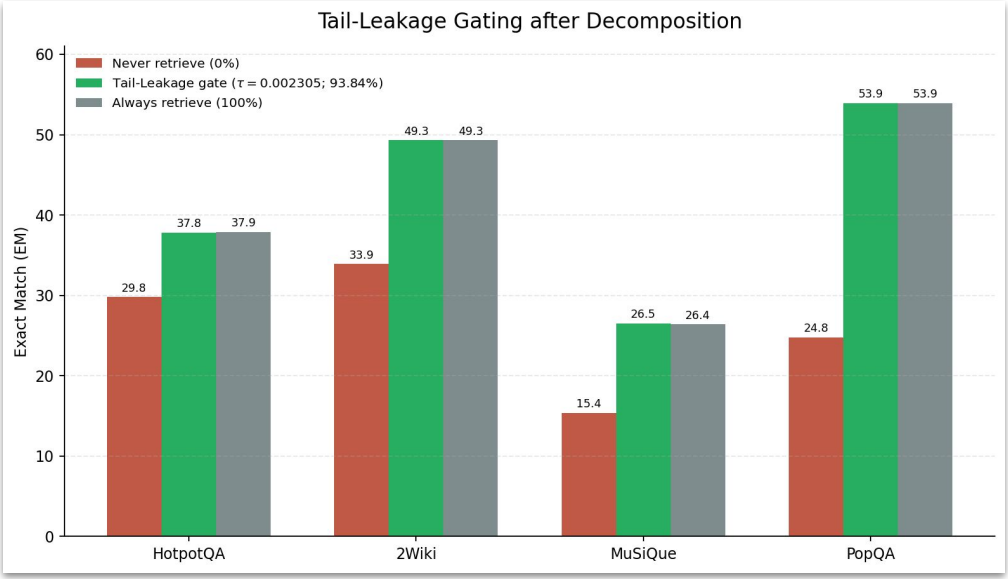
- **Goal**
  - Evaluate whether decomposition improves retrieval quality.
- **Metric**
  - Answer-containing hit@5 , Top-5 retrieved passages contain the gold answer.



Retrieval Is Necessary After Decomposition

Measuring the benefit of retrieval: Parametric-only vs. Gate

- **never-RAG:** decompose only, no retrieval
- **gate:** retrieve only when entropy is high
- **always-RAG:** retrieve at every step



Threshold Sensitivity

τ = 0.00.2305 is selected as the final setting because it gives the best average EM/F1.
Higher τ values reduce retrieval cost with small accuracy drops.

Dataset	τ=0.002305		τ=0.008461		τ=0.044170		τ=0.116500	
	EM	F1	EM	F1	EM	F1	EM	F1
PopQA	**53.90**	**55.73**	53.70	55.53	53.70	55.53	51.90	53.74
HotpotQA	**37.80**	**49.03**	37.50	48.72	36.40	47.33	36.30	47.22
2Wiki	**49.30**	**56.22**	48.70	55.63	47.50	54.57	45.80	52.83
MuSiQue	26.50	34.63	**26.60**	**34.74**	24.80	33.45	23.70	32.69
Average	**41.88**	**48.90**	41.62	48.66	40.60	47.72	39.42	46.62
Avg retrieval rate	93.84%		87.82%		75.51%		64.40%	

Path-aware Title Retrieval

- **Observation**
  - Short sub-questions usually point to a specific entity, so searching the title field helps find the right article.
  - Long original questions contain more words and constraints, so title matching can bring in irrelevant articles.
- **Strategy**
  - Direct path / Original question → Title off
  - Decomposition path / Sub-question → Title on

Dataset	Title-off		Title-on		Path-aware	
	EM	F1	EM	F1	EM	F1
PopQA	51.2	53.5	51.5	53.9	**51.8**	**54.2**
HotpotQA	31.9	41.2	30.5	39.9	31.8	**41.2**
MuSiQue	22.5	30.5	24.5	33.1	**24.6**	**33.5**
2Wiki	45.2	51.3	45.9	52.5	**46.9**	**53.5**

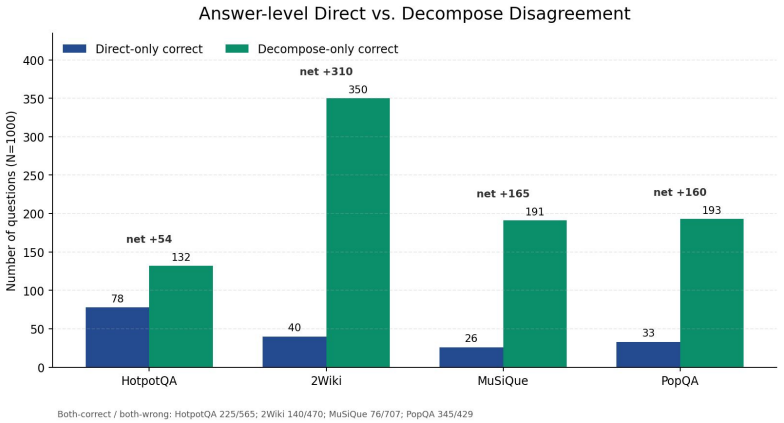
Analysis

- **Direct-vs-decompose routing behavior**
- **Oracle routing gap**

Correct Answer Distribution Across Two Paths

Analyze whether direct answering and decomposition solve different questions.

Dataset	Both Correct	Direct only	Decomposeo nly	Both Wrong
PopQA	345	33	193	429
Hotpot QA	225	78	132	565
2Wiki	140	40	350	470
MuSi Que	76	26	191	707



Verifier Path Selection

The verifier primarily selects decomposition while preserving direct answers when they are more reliable.

Dataset	Choose Direct	Choose Decompose
HotpotQA	17.1%	82.9%
2wiki	2.5%	97.5%
MuSiQue	12.1%	87.9%
PopQA	1.0%	99.0%
Avearge	8.2%	91.8%

Direct-vs-Decompose Oracle Gap

Estimate how much performance can improve with a perfect direct-vs-decompose router.

Dataset	Direct EM	Decomp EM	Current Router EM	Oracle EM	Oracle - Router
PopQA	37.80	54.00	53.90	57.20	+3.30
HotpotQA	30.30	36.00	37.80	43.80	+6.00
2Wiki	18.00	49.70	49.30	53.30	+4.00
MuSiQue	10.20	26.70	26.50	29.40	+2.90
Average	24.08	41.60	41.88	45.93	+4.05

This motivates treating **decomposition as a routing problem**, **not a fixed always-decompose strategy**.

Conclusions

Conclusions

We introduce SEEK, a lightweight adaptive retrieval framework for question answering.

- Proposes tail-leakage surprisal, a new confidence signal that better identifies answer uncertainty than conventional entropy.
- Supports adaptive retrieval across both single-hop and multi-hop questions.
- Applies a training-free verifier to select robust final answers.

Thanks!

Appendix

Gating Improves Efficiency

We compare different retrieval thresholds τ in the final SEER configuration.
The goal is to check whether performance remains stable when the evidence threshold changes.

Dataset	Gate EM	Always RAG EM	Gate - Always	Retrieval rate
PopQA	54.0	54.0	+0.0	97.60%
HotpotQA	36.0	35.9	+0.1	89.23%
2Wiki	49.7	49.7	+0.0	95.75%
MuSiQue	26.7	26.7	+0.0	94.78%

Multi-Step Retrieval and its Cost

Multi-Step Retrieval allows model to iteratively fetch and reason over information, solving accuracy for complex queries.

Benefit

Handles complex, multi-hop questions that require chaining facts across multiple documents.

Cost

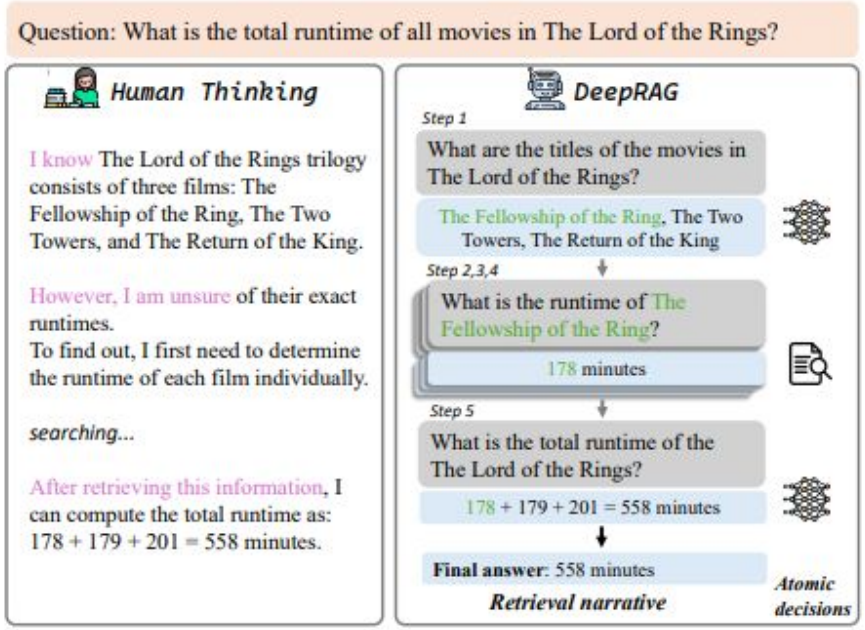
Severe **computational overhead** and **increased latency** if applied indiscriminately to all queries.

It solves the accuracy issue but destroys efficiency if applied blindly - we need a way to selectively decide when retrieval is needed.

[3] Trivedi et al. "Interleaving Retrieval with Chain-of-Thought Reasoning for Knowledge-Intensive Multi-Step Questions"(ACL 2023)

DeepRAG: Two Core Components

- DeepRAG mimics **how humans search the web** — break down a question, decide step-by-step whether to look something up or recall it.



Retrieval Narrative

Structures the workflow: at each step, generate the next sub-query informed by previous answers — keeps the reasoning chain coherent and adaptive.

Atomic Decisions

For each sub-query, the model picks one of two actions: $\delta \in \{\text{retrieve}, \text{parametric}\}$ — driven by self-awareness of its knowledge boundary.

[8] Guan et al. "DEEPRAG: THINKING TO RETRIEVE STEP BY STEP FOR LARGE LANGUAGE MODELS"(ICLR 2026)

Original-question Evidence Union Ablation

Goal : Test whether sub-question retrieval should keep evidence from the original question.

Evidence union :

Retrieve with sub-question + Reuse original-question retrieved documents

→ Merged context

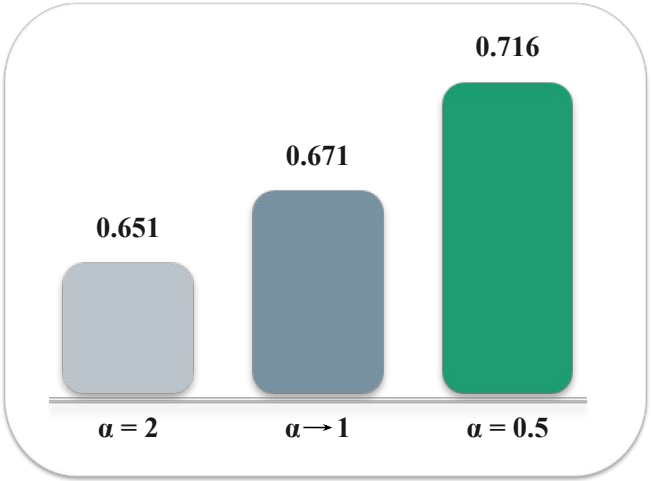
→ Answer the sub-question

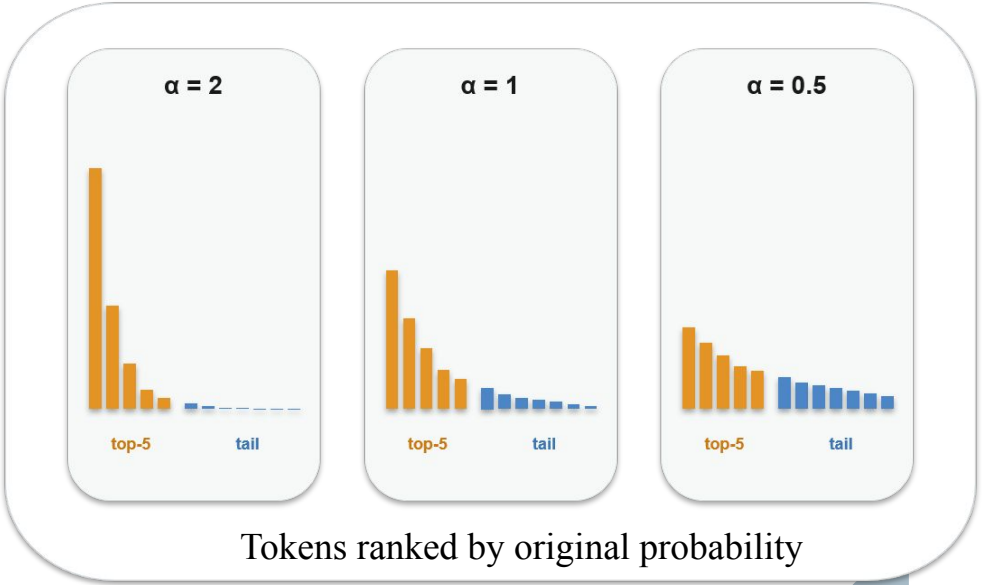
Dataset	Before Evidence union		After Evidence union		Gain	
	EM	F1	EM	F1	EM	F1
PopQA	51.80	54.19	**52.00**	**54.83**	+0.20	+0.64
HotpotQA	31.80	41.25	**34.30**	**44.26**	+2.50	+3.01
2Wiki	46.90	53.51	**49.90**	**56.42**	+3.00	+2.91
MuSiQue	24.60	**33.55**	**24.80**	32.87	+0.20	-0.68
Average	38.78	45.62	**40.25**	**47.09**	+1.47	+1.47

Emphasizing the Tail Improves Error Detection

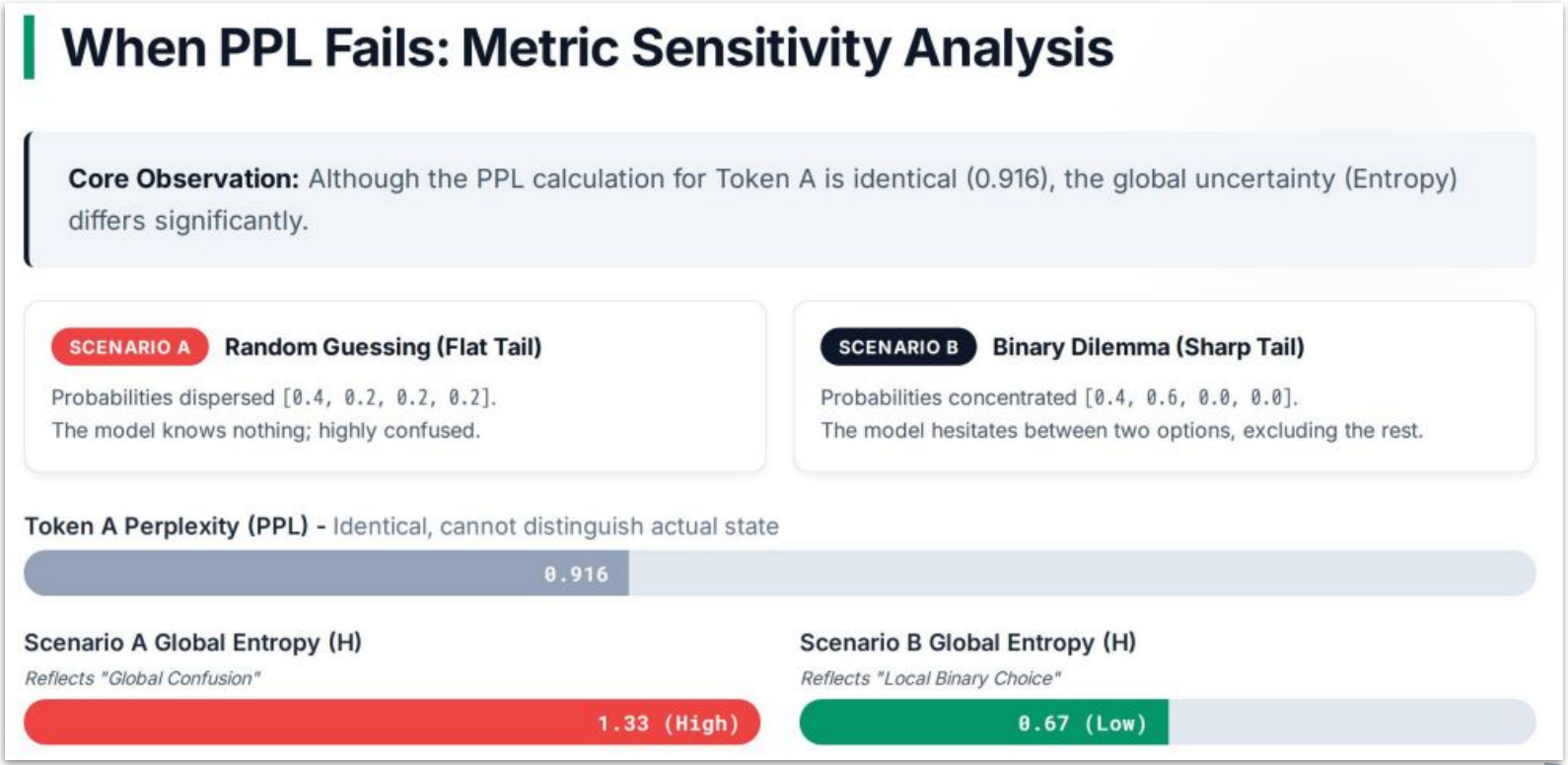
Lower \(\alpha\) gives low-probability tokens more influence, raising 2Wiki AUROC from 0.651 to 0.716.

$$H_\alpha(p) = \frac{1}{1-\alpha} \log \sum_v p(v)^\alpha \qquad H(p) = -\sum_{v \in \mathcal{V}} p(v) \log p(v)$$





Research - Recap



Research - Testing the viability of SC for RAG triggering via multiple methods.

Dataset: HotpotQA 1000Questions (5 no-CoT samples/Questions, T=0.7)

Task: Classify HI_correct vs. HI_wrong

Metric: AUROC

HI_correct (n=161)

The model is naturally competent.

- 5/5 samples are correct.
- **Action:** DO NOT trigger RAG. Save cost/latency.

HI_wrong (n=339)

The model exhibits "Confident Hallucination".

- 5/5 samples are wrong.
- Answers are either consistently wrong or scattered.
- **Action:** MUST trigger RAG.

Research - Evaluating the suitability of different metrics for RAG triggering .

Detecting Confident Hallucinations: Which signal catches **"Stable but Wrong"** generations .

Rank	Feature Name	Location	AUROC
1	Maximum Uncertainty on Answer Entity	first-sentence	0.784
2	Unique Answer Count	multi-sample	0.784
3	Lowest Confidence on Answer Entity	first-sentence	0.781
4	Full-Sequence Self-Certainty	full-answer	0.554
5	Full-Sequence Perplexity	full-answer	0.541

FLARE: Takeaways

- FLARE establishes **active, confidence-driven retrieval** — but its design choices leave open problems that motivate our work.

Strengths

- ✓ **Training-free:** applies to any LM at inference, no fine-tuning required.
- ✓ **Active timing:** retrieval only when the model signals uncertainty.
- ✓ **Transparent signal:** token probability is directly inspectable.

Limitations

- ✗ **Token-level noise:** a single low-prob token (e.g. a connective) can spuriously trigger retrieval.
- ✗ **Wrong granularity:** sentence-level loop is misaligned with sub-question reasoning units.
- ✗ **Threshold-sensitive:** θ requires per-dataset tuning; calibration drifts across LMs.

Takeaway for our work:
we keep the confidence-driven principle, but shift the unit from **token / sentence** to **sub-question**, and replace min-token probability with **entropy over the first-entity span**.

[6] Jiang et al. "Active Retrieval Augmented Generation (FLARE)"(EMNLP 2023)

DeepRAG: Takeaways

- DeepRAG achieves **sub-question-level adaptive retrieval** with strong empirical gains — but its training pipeline is the cost we want to avoid.

Strengths

- ✔ **Right granularity:** decisions made per sub-question, naturally aligned with multi-hop reasoning.
- ✔ **Knowledge calibration:** model learns to distinguish what it knows vs what needs retrieval.
- ✔ **Strong empirical gains:** +21.99% accuracy across 5 QA benchmarks, with fewer retrievals.

Limitations

- ✘ **Expensive data synthesis:** binary tree search must be run for every training sub-query.
- ✘ **Two-stage training:** SFT + DPO pipeline is heavy and tightly coupled to the backbone.
- ✘ **Opaque gating:** atomic decision is learned end-to-end — no inspectable confidence signal.

Takeaway for our work:
DeepRAG validates the **sub-question granularity**, but its gate requires **tree search + SFT + DPO**. We replace this with a **training-free entropy signal on the first-entity span**.

Synthesis: bridges DeepRAG to our SubQ-Entropy proposal (same granularity, training-free gate).

Adaptive-RAG: Takeaways

- Adaptive-RAG is the cleanest formulation of **query-level routing** — but its decision granularity stops at the question, leaving no room for mid-reasoning adjustment.

Strengths

- ✔ **Lightweight router:** T5-Large adds negligible latency vs full-pipeline cost.
- ✔ **No human labels:** training data is collected automatically from model outcomes + dataset bias.
- ✔ **Cost-aware:** saves compute on simple queries by skipping retrieval entirely.

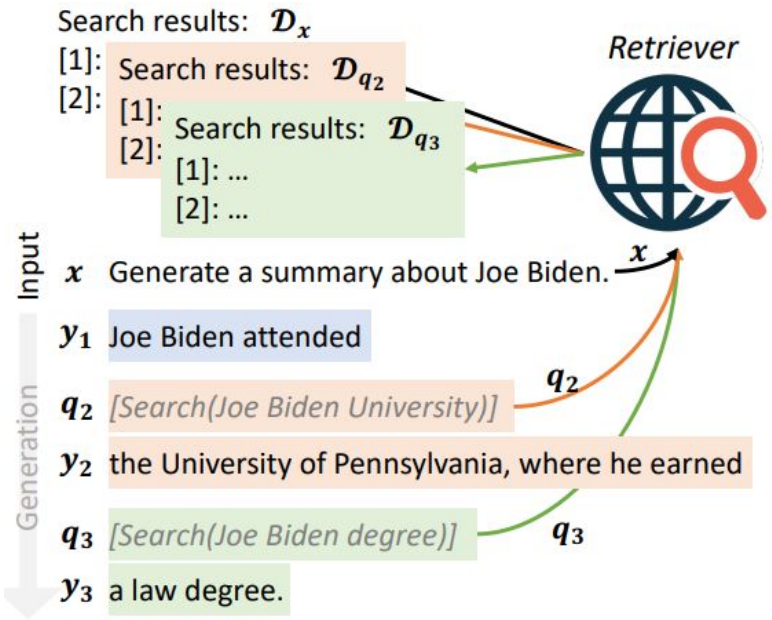
Limitations

- ✘ **Question-level granularity:** decides once on the whole query — cannot react to mid-reasoning uncertainty.
- ✘ **Surface-feature routing:** uses query text alone, not the model's actual confidence on each step.
- ✘ **Cascade error risk:** a misclassified "easy" multi-hop query gets under-retrieved with no recovery.

Takeaway for our work:
we keep the spirit of adaptive routing but shift the decision from **whole-question surface features** to **per-sub-question model confidence** — so the system can recover within a chain instead of committing at the start.

FLARE-instruct: A Variant

- Instead of probing token probabilities, FLARE-instruct asks the LM to **emit a search token** when it wants to retrieve — inspired by Toolformer.



How it works

Trigger	LM generates a special [Search(query)] token mid-decoding.
Mechanism	Generation pauses, retriever runs with the emitted query, result is fed back, decoding resumes.
Prompting	Few-shot retrieval-encouraging instructions teach the LM when to call search.
Pros	Query is fully natural-language, intent-aware.
Cons	Depends on the LM's ability to follow retrieval instructions — weaker LMs ignore the prompt.

[6] Jiang et al. "Active Retrieval Augmented Generation (FLARE)"(EMNLP 2023)

MDP Formulation

- DeepRAG formalizes retrieval-augmented reasoning as an **MDP ⟨S, A, P, R⟩** — every step is a state, every retrieval choice is an action.

MDP components	
S — State	Original question + all sub-queries and intermediate answers so far.
A — Action	Two atomic choices per step: σ (terminate? next sub-query) and δ ∈ {retrieve, parametric}.
P — Transition	If σ = terminate → emit final answer; else apply δ to fetch the next intermediate answer.
R — Reward	+1 if final answer correct; –cost per retrieval to discourage unnecessary lookups.

① Binary Tree Search

Each sub-query expands two children: retrieve vs use parametric memory. The tree explores reasoning paths until the answer is correct; the optimal path = correct answer with the fewest retrievals.

② Imitation Learning (SFT)

Distill the optimal paths into supervised data; fine-tune the LM to imitate "sub-query → atomic decision → intermediate answer → … → terminate".

③ Chain of Calibration (DPO)

Use tree search again to construct preference pairs over atomic decisions; apply DPO so the model retrieves only when its parametric memory genuinely cannot answer.

[8] Guan et al. "DEEPRAG: THINKING TO RETRIEVE STEP BY STEP FOR LARGE LANGUAGE MODELS"(ICLR 2026)

Tail Leakge as a Confidence Signal

$$
\begin{aligned}
H(p) &= -\sum_{v \in \mathcal{V}_{\text{top5}}} p(v) \log p(v) - \sum_{v \in \mathcal{V}_{\text{tail}}} p(v) \log p(v) \\
&= -\sum_{v \in \mathcal{V}_{\text{top5}}} m_5 p_{\text{top5}}(v) \log \left[m_5 p_{\text{top5}}(v) \right] \\
&\quad -\sum_{v \in \mathcal{V}_{\text{tail}}} (1 - m_5) p_{\text{tail}}(v) \log \left[(1 - m_5) p_{\text{tail}}(v) \right] \\
&= -m_5 \log m_5 - (1 - m_5) \log (1 - m_5) + m_5 H(p_{\text{top5}}) + (1 - m_5) H(p_{\text{tail}}) \\
&= H_{\text{B}}(m_5) + m_5 H(p_{\text{top5}}) + (1 - m_5) H(p_{\text{tail}})
\end{aligned}
$$

Aggregating Leakage Across the Answer

Sum $-\log m_{\text{head}}^{(t)}$ across tokens to capture small leaks distributed throughout the answer.

$$
\begin{aligned}
H(p) &= -\sum_{v \in \mathcal{V}} p(v) \log p(v). \\
&= -m_{\text{head}} \log m_{\text{head}} - (1 - m_{\text{head}}) \log (1 - m_{\text{head}}) \\
&\quad + m_{\text{head}} H(p_{\text{head}}) + (1 - m_{\text{head}}) H(p_{\text{tail}})
\end{aligned}
$$

$$
\longrightarrow \; -\log m_{\text{head}}^{(t)}
$$

$$
\text{TailLeakage}(y) = \sum_{t=1}^{T} -\log m_{\text{head}}^{(t)}
$$

Tail Leakge as a Confidence Signal

Method - Entropy-Gated Retrieval(這個例子不好)

Entropy measures uncertainty over the full token distribution, not only the probability of the generated token.

